Lab Assignment 2

Three-Dimensional Rotations and Their Representations

Maths II – Bachelor's Degree in Video Game Design and Development
CITM - UPC

Autumn 2026

Abstract

This assignment extends the vector and matrix algebra developed in lab 1 to three-dimensional rotations. Students implement conversions between rotation matrices, axis-angle descriptions, intrinsic Euler ZYX angles, and quaternions. They also rotate vectors and measure the arithmetic cost of selected operations. The goal is to connect geometric definitions with reliable C++ implementations, including representation ambiguities, singular cases, and input validation. An interactive app and automated tests provide complementary validation.

Repository: `https://github.com/jvllahi/math_2_lab`

Scope and Conventions

All work is in the `lab2/` project. Implement the TODO methods in `assignment/Matrix3x3.cpp` and `assignment/Quat.cpp`; their headers define the public contracts. The generic algebra and supplied helper functions are not new student exercises.

- Rotations are active and right-handed, acting on column vectors. Matrix entries are stored in row-major order.
- Function angles are in radians; the app displays and accepts degrees.
- Euler angles follow intrinsic ZYX, with yaw about Z, pitch about the local Y axis, and roll about the local X axis.
- Quaternion components are ordered `(s,x,y,z)`: scalar first.
- Rotation checks use absolute tolerance 10^{-6} for orthogonality and determinant. The app accepts numerical output error up to $5 \cdot 10^{-6}$; automated tests also check stricter tolerances for selected cases.

The following equations describe the mathematical objects and conventions. Deriving the conversion algorithms and handling their numerical edge cases are part of the assignment; this handout does not provide solution code or pseudocode.

Building on Lab 1

`algebra/LinAlgebra.hpp` and `algebra/LinAlgebra.cpp` provide the generic `Vector`, `Matrix`, and `Operations` from lab 1. `Vec3` contains a three-element `Vector`; `Matrix3x3` contains a 3×3 `Matrix`. Their fixed-size arithmetic operators reuse the generic implementation.

The supplied `helpers/` include matrix allocation and row swaps, the Euclidean norm, fixed-size construction and access, basic geometry, quaternion normalization, and independent validation. Use vector indices $0, 1, 2$ and `At(row,col)` for matrix entries. Default vectors and matrices are zero; `Matrix3x3::Identity()` constructs the identity.

Why use algebraic expressions?

Before writing component-by-component loops, consider whether the matrix and vector operators from lab 1 can express the algebra you need. They are also available for `Matrix3x3` and `Vec3`, with the same natural syntax.

Reusing these operations can keep your code closer to the mathematical notation, reduce repetitive arithmetic, and make it easier to read and check. Choosing how to apply them to the rotation methods is part of the assignment.

Exercise 1: Rotation Matrices and Axis-Angle

a) Mathematical Summary

A proper rotation matrix belongs to

$$SO(3) = \{R \in \mathbb{R}^{3\times 3} : R^{\mathsf{T}}R = I,\ \det R = 1\}.$$

Its columns form a right-handed orthonormal basis. A rotated vector is $\boldsymbol{v}' = R\boldsymbol{v}$, and rotation preserves lengths and dot products. Orthogonality alone also permits reflections, which have determinant -1.

An axis-angle description consists of a unit axis $\boldsymbol{u}$ and a physical rotation angle θ. Rodrigues' geometric relation is

$$R\boldsymbol{v} = \boldsymbol{v}\cos\theta + (\boldsymbol{u}\times\boldsymbol{v})\sin\theta + \boldsymbol{u}(\boldsymbol{u}\cdot\boldsymbol{v})(1-\cos\theta).$$

The axis is unchanged by the rotation. Equivalent descriptions are not unique: angles can differ by full turns, and reversing both axis and angle describes the same rotation. For output, use a unit axis and $\theta \in [0, \pi]$. The identity has no unique axis; at a half-turn, opposite axes are equivalent. These ambiguities must not be confused with incorrect rotations.

b) Implementation

Implement in `assignment/Matrix3x3.cpp`:

- `IsRotation`: return whether the matrix is a proper rotation within the specified tolerance. Non-finite entries return `false`.
- `RotationAxisAngle`: construct a rotation from an axis and an angle. Normalize the finite, nonzero axis; reject a zero axis even when the angle is zero.
- `ToAxisAngle`: extract a unit axis and an angle in $[0, \pi]$. Handle identity, half-turns, and nearby angles reliably.
- `Rotate`: rotate a finite vector using a valid rotation matrix and accumulate the scalar arithmetic counts of the product kernel.

Except for the boolean `IsRotation` check, invalid matrix, axis, vector, or angle inputs must produce `std::invalid_argument`. Choose a deterministic unit axis for identity output and explain the choice.

c) Report

(a) Why must a proper rotation satisfy both orthogonality and determinant +1? Provide a matrix that satisfies one condition but is not a rotation.

(b) Explain geometrically why rotation preserves vector length and leaves its axis unchanged. Show numerical evidence from one test.

(c) Why are identity and half-turn rotations special for axis-angle extraction? Explain the ambiguities and how your implementation handles them.

(d) Report one normal and one near-half-turn round trip. State the input, the returned parameters, and the reconstructed matrix error.

Exercise 2: Intrinsic Euler ZYX Angles

a) Mathematical Summary

Euler angles describe orientation using three elementary rotations. This assignment fixes the convention

$$R(\psi, \theta, \phi) = R_z(\psi)R_y(\theta)R_x(\phi),$$

where ψ is yaw, θ is pitch, and ϕ is roll. The matrices R_x, R_y, and R_z are right-handed rotations about the corresponding coordinate axes. Matrix multiplication is not generally commutative, so the axis order and the active column-vector convention matter.

Different Euler triples can represent the same orientation. Canonical outputs must satisfy

$$\psi, \phi \in [-\pi, \pi], \qquad \theta \in [-\pi/2, \pi/2].$$

At $\theta = \pm\pi/2$, two rotational degrees of freedom become coupled: this is gimbal lock. A unique yaw/roll pair cannot be recovered. For a reproducible output convention in this assignment, set roll to zero at exact gimbal lock and return an equivalent orientation.

b) Implementation

Implement in `assignment/Matrix3x3.cpp`:

- `FromEulerZYX`: construct the specified orientation from finite yaw, pitch, and roll inputs in radians.
- `ToEulerZYX`: extract canonical Euler angles from a valid rotation matrix, including both signs of gimbal lock and nearby cases.

Reject non-finite input angles or non-rotation matrices with `std::invalid_argument`. Validate equivalent orientations rather than requiring an extracted Euler triple to equal a noncanonical input triple.

c) Report

(a) Explain the chosen intrinsic ZYX convention. Why would changing the multiplication order generally change the orientation?

(b) What is lost at gimbal lock? Explain why different yaw/roll pairs can describe the same orientation, and state your output convention.

(c) Compare a normal round trip with pitch at $+90°$, $-90°$, and close to either value. Include reconstructed matrix errors.

(d) Give two distinct Euler triples representing the same orientation. Explain why comparing angles directly is not a sufficient correctness test.

Exercise 3: Quaternions and Rotation

a) Mathematical Summary

A quaternion has a scalar part and a three-dimensional vector part:

$$q = s + x\,i + y\,j + z\,k = (s, \boldsymbol{u}), \qquad \boldsymbol{u} = (x, y, z).$$

The units satisfy $i^2 = j^2 = k^2 = ijk = -1$. For $p = (s, \boldsymbol{u})$ and $q = (t, \boldsymbol{w})$, the Hamilton product is defined by

$$pq = (st - \boldsymbol{u} \cdot \boldsymbol{w},\ s\boldsymbol{w} + t\boldsymbol{u} + \boldsymbol{u} \times \boldsymbol{w}).$$

It is associative but not generally commutative.

The norm and conjugate are

$$\|q\| = \sqrt{s^2 + x^2 + y^2 + z^2}, \qquad q^* = (s, -x, -y, -z).$$

Unit quaternions encode rotations. The axis-angle representation corresponds to

$$q = \left(\cos\frac{\theta}{2},\ \boldsymbol{u}\sin\frac{\theta}{2}\right), \qquad \|\boldsymbol{u}\| = 1.$$

For a unit quaternion, the vector part of $q(0, \boldsymbol{v})q^*$ is the rotated vector. The quaternions q and $-q$ represent the same rotation. The product $q_1 q_2$ composes rotations with q_2 acting first.

b) Implementation

Implement in `assignment/Quat.cpp`:

- `Multiply`: compute the Hamilton product without normalizing either operand, and accumulate scalar arithmetic counts. Finite zero quaternions are valid operands for this algebraic operation.
- `Rotate`: normalize the quaternion, rotate a finite vector, and accumulate only the rotation kernel's scalar arithmetic counts.
- `FromAxisAngle` and `ToAxisAngle`: convert between unit quaternions and axis-angle descriptions with the output conventions of exercise 1.
- `FromMatrix3x3` and `ToMatrix3x3`: convert between unit quaternions and proper rotation matrices.

`Normalized` is supplied in the helpers. Rotation and conversion methods must normalize finite, nonzero quaternion inputs; zero quaternions do not describe rotations. Axis inputs must be finite and nonzero; angle inputs must be finite. Invalid inputs throw `std::invalid_argument`. Matrix-to-quaternion and axis-to-quaternion outputs must have unit norm. Axis-angle extraction must treat q and $-q$ consistently.

c) Report

(a) Explain the role of the half-angle and why q and $-q$ describe the same rotation. Demonstrate the latter numerically.

(b) Give a pair of quaternion rotations whose composition changes when the operands are swapped. Relate the result to matrix composition.

(c) Why is normalization appropriate for rotation/conversion methods, but not for an arbitrary Hamilton product? What happens with a zero quaternion?

(d) Compare rotating the same vector using a matrix and a quaternion. Report output error and each arithmetic count, and justify any difference from the app's advisory standard counts.

(e) Demonstrate a matrix–quaternion–matrix round trip for a normal rotation and a half-turn. Explain why quaternion components alone are not a sufficient equivalence check.

Validation and Arithmetic Counts

Use `Operations` to accumulate scalar additions, subtractions, multiplications, and divisions in `Matrix3x3::Rotate`, `Quat::Multiply`, and `Quat::Rotate`. Do not reset a counter provided by the caller.

Exclude index arithmetic, comparisons, control flow, input validation, normalization, and independent reference calculations. The ordinary overloaded algebra operators do not update these counters automatically. Account for the arithmetic actually performed by the generic operations you call, including additions to a zero accumulator where applicable.

The app's count comparison is advisory, separate from numerical correctness. Explain your counting model and any alternative algorithm in the report. Elapsed time is informational, not a standalone proof of correctness.

Use round trips and geometric invariants to compare orientations. A small error in recovered parameters is not always meaningful near a non-unique representation. Report the error in reconstructed matrices or rotated vectors, and verify the required output ranges and unit norms separately.

How to Test the Implementation

Run the supplied presets from the `lab2/` folder:

```
cmake --preset default-tests
cmake --build --preset default-tests --parallel
ctest --preset default-tests
```

The test executable is `build-tests/bin/lab2_tests`, with `.exe` on Windows. Tests need no display. Unfinished rotation TODOs are expected to fail; the supplied geometry tests should already pass. Add focused tests for your own edge cases rather than relying only on the provided examples.

Also build and run the interactive app:

```
cmake --preset default
```

```
cmake --build --preset default --parallel
./build/bin/lab2
```

On Windows PowerShell, run `.\build\bin\lab2.exe` instead. Use **Operation**, **Example**, **Load example**, and **Run operation**. Check status and details, numerical output, error, and applicable counts. A rejected invalid input can correctly produce no numerical output. See `README.md` for the full UI workflow.

Case	What to verify
Normal rotations	Known orientations, vector length, valid matrix, and round trips.
Identity	Equivalent identity orientation and valid, deterministic output parameters.
Half-turn and nearby angles	Stable axis-angle and quaternion conversions with unit outputs.
Gimbal lock and nearby pitch	Both signs, canonical Euler ranges, and equivalent reconstructed rotation.
Quaternion sign and scale	Equivalent rotation for opposite signs and nonzero scaled inputs.
Invalid inputs	Reflection/non-rotation checks; zero axis or quaternion, and non-finite values rejected where required.

Record reproducible inputs, expected properties, actual results, and tolerances. Include automated test results and at least one student-authored edge case for each exercise in your report.

Evaluation Criteria

The same weighting structure as lab 1 applies:

- Exercise 1 (35%): Implementation 25% + Report 10%.
- Exercise 2 (35%): Implementation 25% + Report 10%.
- Exercise 3 (30%): Implementation 20% + Report 10%.

Evaluation considers mathematical correctness, adherence to the API contracts, singular-case handling, validation, justified arithmetic counts, reproducible tests, and the explanations requested in each report section.

The assignment is optional. If it is not submitted, the final lab grade is the lab exam grade. If it is submitted, the final lab grade is the greater of the exam grade and the average of the assignment and exam grades, so submission does not penalize the student.

The code must work correctly; otherwise, the corresponding exercise receives zero. If plagiarism is detected, the final grade is zero.

Academic Integrity and AI Use

Using AI tools (LLMs, code assistants) as tutoring support is allowed. Submitting generated code or report text as your own work without disclosure is not allowed.

- If AI support was used, include `AI_USAGE.md` in the submitted zip.
- List the tool name, what it was used for, and the final prompts that directly influenced the submitted code or report.
- You must be able to explain and justify every submitted function and test.

During review, students will be asked to explain selected code fragments or test decisions. A submitted part that a student cannot explain will be considered invalid for grading.

Submission

Groups may contain up to three people. Submit through ATENEA using the naming pattern below, omitting unused member slots for smaller groups:

`Surname1Name1_Surname2Name2_Surname3Name3_Lab2.zip`

Include:

- A buildable `lab2/` project preserving its relative folder structure: completed assignment sources and headers, provided `algebra/`, `helpers/`, and `app/`, tests, and CMake files. Include any student-authored tests and necessary build changes.
- A PDF report of 2–4 pages, with a section for each exercise answering its report questions. Identify the group members and include numerical evidence, count comparisons, test results, and conclusions.
- `AI_USAGE.md`, if applicable.

Do not include the provided `solution/` folder, build directories, executables, or downloaded dependencies. Your submitted implementation must build with the default assignment preset from a fresh project directory. The submission is evaluated jointly with the group members; the lab exam is individual.

Important Dates

Lab 2 Assignment Submission Deadline: To be announced.
Term Exam: To be announced.
Lab Practical Exam: To be announced.